

Physical Units Expression Language

PUEL Specification for SciNumTools v3

Ondrej Pego Jaura

September 2026

Contents

1	PUEL Specification	1
1.1	Overview	1
1.2	Design Principles	1
1.2.1	Relationship to Existing Unit Standards	2
1.3	Unit Model	2
1.3.1	Base units	2
1.3.2	Scaling factors	2
1.3.3	Named Entities	2
1.4	Unit expressions	3
1.4.1	General Form	3
1.4.2	Expression Parsing	4
1.4.3	Canonical Representation and Serialization	4
1.5	Conversions	4
1.6	Appendinx A: Tables	4
1.6.1	Scaling prefixes	4

1 PUEL Specification

1.1 Overview

PUEL (Physical Units Expression Language) is a domain-specific language for representing and manipulating physical units and dimensional expressions.

It defines the syntax and structure of unit expressions used in scientific computations within the SciNumTools3 framework. Evaluation, normalization, dimensional analysis, and unit conversion are performed by the underlying EXS and PUQ modules.

The language is designed for the following use cases: * As a component of DIPL for defining and validating unit-aware parameters. * As a standalone language for the concise and machine-readable representation of physical unit expressions. * As a backend for programmatic interfaces, such as Python bindings and other language integrations.

1.2 Design Principles

PUEL is designed to be:

- **Concise** — compact textual representation of units
- **Composable** — units can be combined using algebraic operations
- **Deterministic** — expressions resolve to a canonical dimensional form
- **Extensible** — supports custom units and prefixes
- **Solver-backed** — evaluation delegated to EXS

1.2.1 Relationship to Existing Unit Standards

PUEL is designed as a machine-readable language for representing physical quantities and unit expressions, rather than as a replacement for established unit coding systems. In particular, PUEL is intended to be interoperable with standards and vocabularies such as the International System of Units (SI), the Unified Code for Units of Measure (UCUM), UDUNITS, QUDT, and the OM ontology. PUEL extends the scope of conventional unit notations by providing a unified syntax for numerical values, units, uncertainties, physical constants, symbolic quantities, and algebraic expressions. Where applicable, PUEL unit expressions **SHOULD** be convertible to and from established unit representations, while the semantic interpretation of a PUEL expression remains independent of any particular serialization format. This design allows PUEL to serve as a compact interchange notation for scientific quantities while remaining compatible with existing unit standards and ontologies.

1.3 Unit Model

1.3.1 Base units

All unit expressions are ultimately represented in terms of a fixed set of base dimensions:

- Length in meters (**m**)
- Mass in grams (**g**)
- Time in seconds (**s**)
- Temperature in kelvins (**K**)
- Electric charge in coulombs (**C**)
- Luminous intensity in candelas (**cd**)
- Amount of substance in moles (**mol**)
- Angle in radians (**rad**)

Derived units are represented as combinations of these base dimensions together with an associated scaling factor.

The choice of base dimensions is driven by implementation considerations and does not exactly match the SI base-unit system.

- Mass is represented using grams (**g**) rather than kilograms (**kg**) to avoid embedding a prefix in a base unit.
- Electric charge is represented using coulombs (**C**) rather than amperes (**A**), making current a derived quantity (**C/s**). This choice defines the electrical dimension in terms of the fundamental conserved quantity of charge rather than the derived quantity of current.
- Angle is included as a base dimension to enable dimensional validation of mathematical functions that require angular arguments. However, angular units **MUST** remain convertible to dimensionless units, reflecting their dimensionless nature in the SI system.

1.3.2 Scaling factors

Scaling prefixes provide a compact way to represent decimal multiples and submultiples of units. A scaling prefix modifies a unit identifier by applying a fixed numerical factor while preserving the unit's dimensionality.

For example, the expressions **m**, **cm**, and **km** all represent units of length and differ only in their scaling factors relative to the base unit.

PUEL defines a standard set of scaling prefixes. Implementations **MUST** recognize and correctly apply these prefixes when parsing unit expressions. However, prefixes **SHOULD** only be permitted for units for which their use is conventional or widely accepted.

The complete list of supported scaling prefixes is provided in Appendix A.1: Scaling Prefixes.

1.3.3 Named Entities

Named entities in PUEL include all units (e.g., Joule **J**, degree Celsius **Ce1**, decibel **dB**), physical constants (e.g., proton mass **{m_p}**, Avogadro constant **{N_A}**, solar luminosity **{L_sol}**), physical quantities (e.g., energy **<E>**, electric flux **<Phi_E>**, radiation dose **<D_r>**), and unit-system scaling factors (e.g., **|E|**, **|Phi_E|**, **|D_r|**). The dimensionality of these entities is defined in terms of the base dimensions described in the previous section.

PUEL distinguishes between these categories using the following notation:

- Units are represented by their symbols: J, Cel, dB
- Physical constants are enclosed in curly braces: {m_p}, {N_A}, {L_sol}
- Physical quantities are enclosed in angle brackets: <E>, <Phi_E>, <D_r>
- Unit-system scaling factors are enclosed in pipe symbols: |E|, |Phi_E|, |D_r|

Unit-system scaling factors represent the numerical coefficients required to convert a physical quantity between the SI system and another system of units. They are primarily intended to support conversions between systems with different dimensional conventions, such as SI and CGS.

Each implementation **MUST** define the numerical values and dimensional representations of all supported derived entities. These definitions **SHOULD** be precomputed and stored to avoid repeated evaluation and dimensional derivation during the processing of unit expressions.

Within a given implementation and unit system, all supported identifiers and scaling-prefix combinations **MUST** be unique. Consequently, every valid token in a PUEL expression **MUST** have exactly one interpretation.

1.4 Unit expressions

1.4.1 General Form

A unit expression consists of an optional unit system identifier, an numerical value, and a unit expression.

General form:

[<system>_] [<value>*] [<unit-expression>]

where:

- <system> is a unit system identifier.
- <value> is a numerical value, optionally including an uncertainty.
- <unit-expression> is a combination of unit identifiers and algebraic operators.

At least one of <value> or <unit-expression> **MUST** be present.

Examples:

m
2.5*m
3.452(3)*kg*m/s2
US_lb*ft
SI_9.81*m/s2

The following rules apply in unit expressions:

Rule	Examples
Expressions MUST NOT contain whitespace; all tokens are written without spaces.	kg*m/s2
Unit identifiers MAY be combined using multiplication (*), division (/), and grouping parentheses ((,)).	kg*mm/skg*m2/(sr*s2)
Exponents are written directly after the corresponding unit identifier and do not use exponentiation operators such as ^ or **.	m2
Fractional exponents are written using the numerator:denominator notation.	kg3:2
Negative exponents are written using a leading minus sign.	s-1
A unit system MAY be specified at the beginning of an expression and is separated from the remainder of the expression by an underscore (_).	US_lb*ft
Numerical values MAY be included in expressions.	2.34e3*mol-9.81*m/s2
The symbols + and - denote the sign of a numerical value or exponent; they do not represent addition or subtraction operators.	+5*m-5*ms-2
Uncertainties are written in parentheses immediately following the last decimal digit of a numerical value. The number of digits in the uncertainty corresponds to the same number of least significant digits in the value.	3.45234(2)e3 → 3452.34 ± 0.023.45234(12) → 3.45234 ± 0.0001212.3(4)e-2 → 0.123 ± 0.004120(5) → 120 ± 5

1.4.2 Expression Parsing

Although numerical values are typically written at the beginning of an expression, they are not restricted to that position. Numerical factors MAY appear anywhere within a unit expression as multiplicative constants. For example, $2\text{kg}\cdot\text{m}/\text{s}^2$ and $\text{kg}\cdot(2\text{m})/\text{s}^2$ are equivalent.

Implementations SHOULD accept numerical factors in any valid position within a unit expression. However, producers of unit expressions SHOULD place any numerical factor at the beginning of the expression, as this improves readability and simplifies parsing.

Unit expressions MAY use division operators and grouping parentheses for readability. For example, $\text{kg}\cdot\text{m}^2/(\text{sr}\cdot\text{s}^2)$ and $\text{kg}\cdot\text{m}^2\cdot\text{sr}^{-1}\cdot\text{s}^{-2}$ represent the same unit expression.

1.4.3 Canonical Representation and Serialization

Implementations SHOULD internally normalize unit expressions to a canonical form in which all unit factors appear in the numerator and division is represented using negative exponents. For example, the expression $\text{kg}\cdot\text{m}^2/(\text{sr}\cdot\text{s}^2)$ is canonically represented as $\text{kg}\cdot\text{m}^2\cdot\text{sr}^{-1}\cdot\text{s}^{-2}$.

The canonical form provides a unique and deterministic textual representation of a unit expression. Because the original placement of division operators and grouping parentheses cannot, in general, be reconstructed from a normalized expression, implementations SHOULD use the canonical form when serializing unit expressions.

If a unit expression contains a numerical factor, serialized expressions MUST place the numerical factor at the beginning of the expression.

When representing base dimensions, only dimensions with non-zero exponents SHOULD be included. The order of the base dimensions MUST be preserved as specified in Section 3.1..

1.5 Conversions

PUEL supports conversion between units of identical dimensions:

$\text{km} \rightarrow \text{m}$
 $\text{eV} \rightarrow \text{J}$
 $\text{dB} \rightarrow \text{W}$

Special handling exists for:

- logarithmic units
- temperature units

These require custom conversion rules.

1.6 Appendix A: Tables

1.6.1 Scaling prefixes

Prefix	Factor	Name
Y	1e24	yotta
Z	1e21	zetta
E	1e18	exa
P	1e15	peta
T	1e12	tera
G	1e9	giga
M	1e6	mega
k	1e3	kilo
h	1e2	hecto
da	1e1	deka
d	1e-1	deci
c	1e-2	centi
m	1e-3	milli
u	1e-6	micro
n	1e-9	nano
p	1e-12	pico

Prefix	Factor	Name
f	1e-15	femto
a	1e-18	atto
z	1e-21	zepto
y	1e-24	yocto

[!NOTE] The prefix **u** is used to represent micro (μ) in ASCII-only unit expressions.